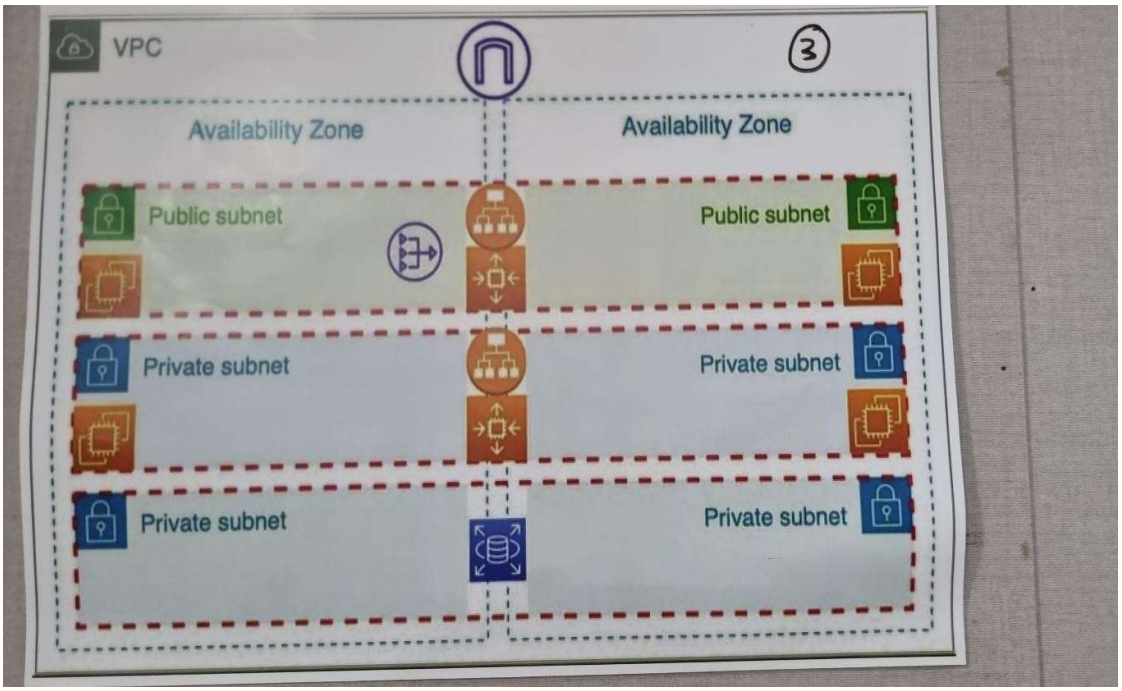
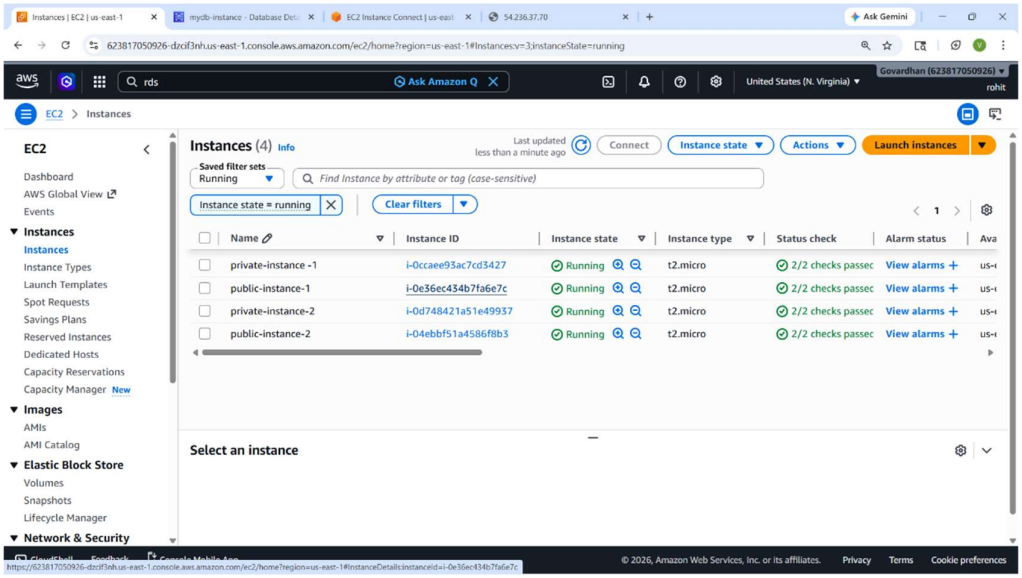
Project :- Terraform

Name :- k.Veera Bhaskara Durga Prasad

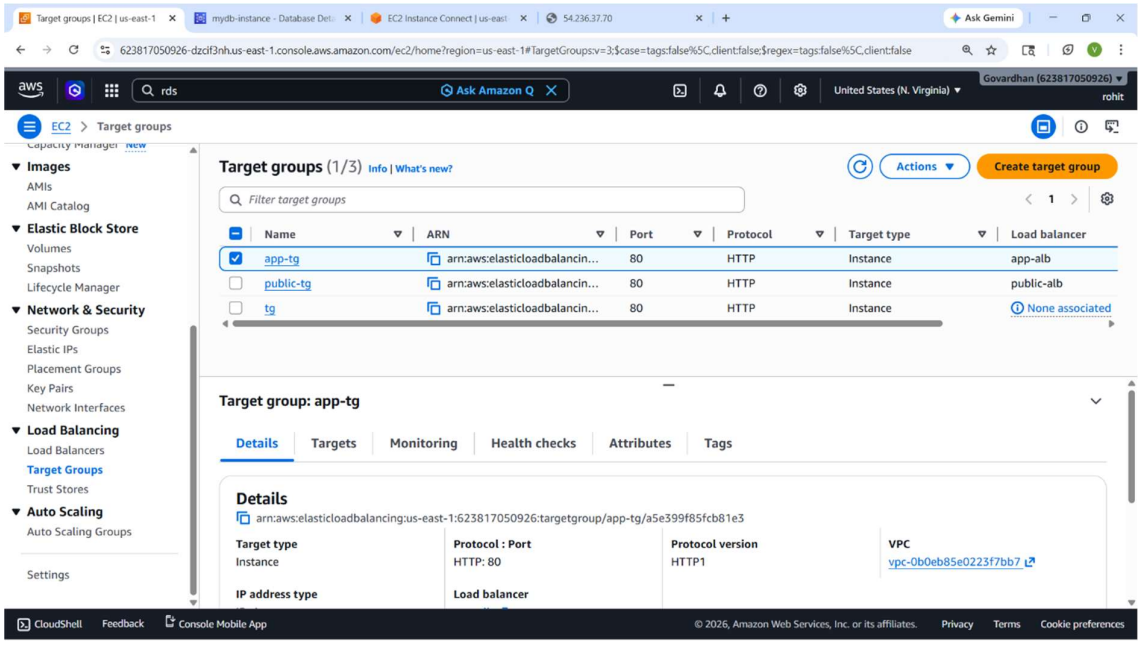
Email :- kpurushotham9000@gmail.com





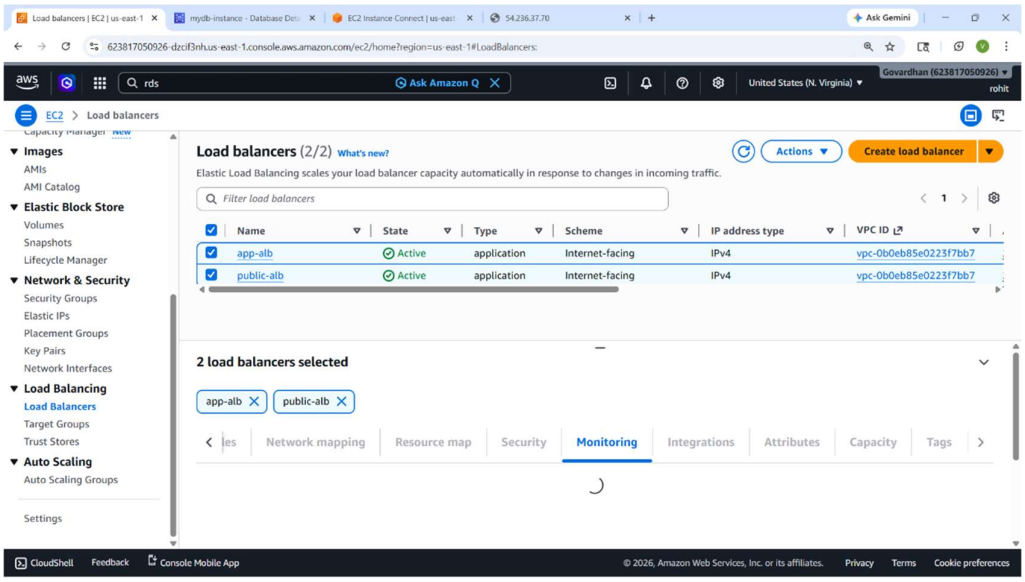
Step 1: Provider Configuration

Terraform is initialized with the required AWS provider.
The region is set to us-east-1, where all resources will be provisioned.
This allows Terraform to authenticate and interact with AWS.
Version control ensures compatibility with the correct provider version.



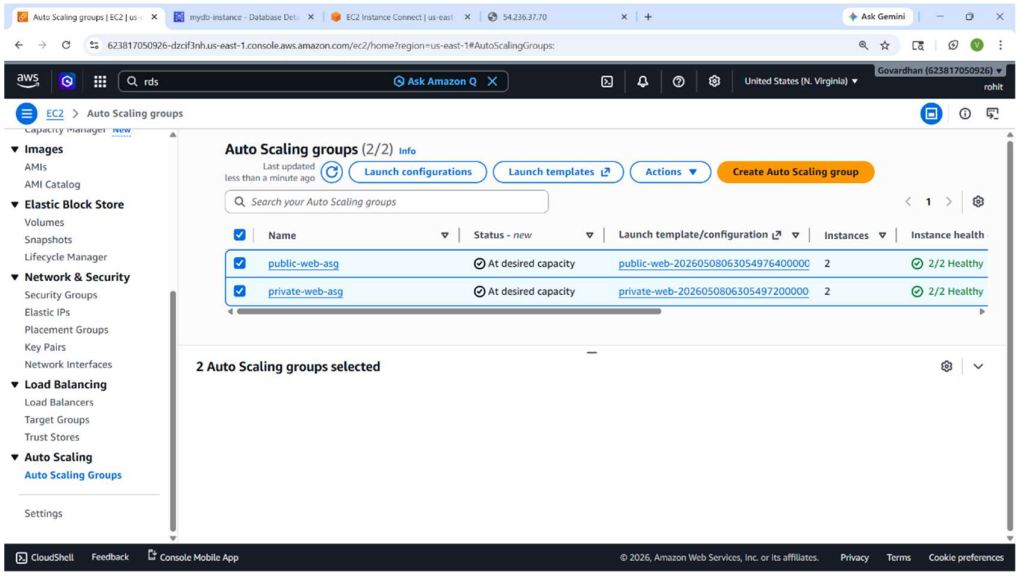
Step 2: VPC Creation

A Virtual Private Cloud (VPC) is created with a CIDR block of 10.0.0.0/16.
DNS hostnames and DNS support are enabled for internal resolution.
This VPC serves as the isolated network environment for all services. Tags are used for easy identification and resource tracking.



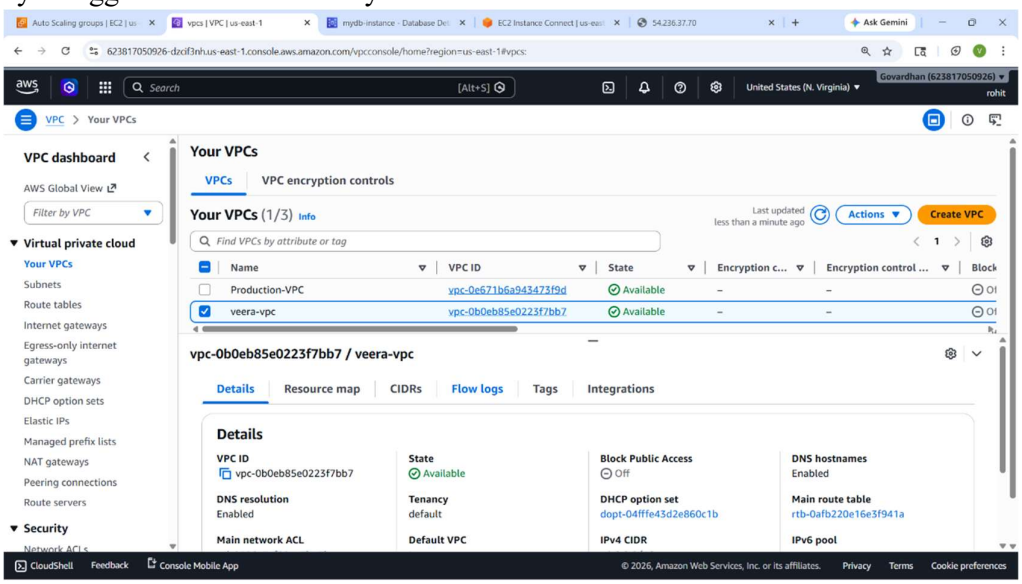
Step 3: Subnet Creation

Six subnets are created across two availability zones for high availability.
These include two public, two private EC2, and two private RDS subnets.
Public subnets are internet-facing and allow public access.
Private subnets are isolated for internal workloads and databases.



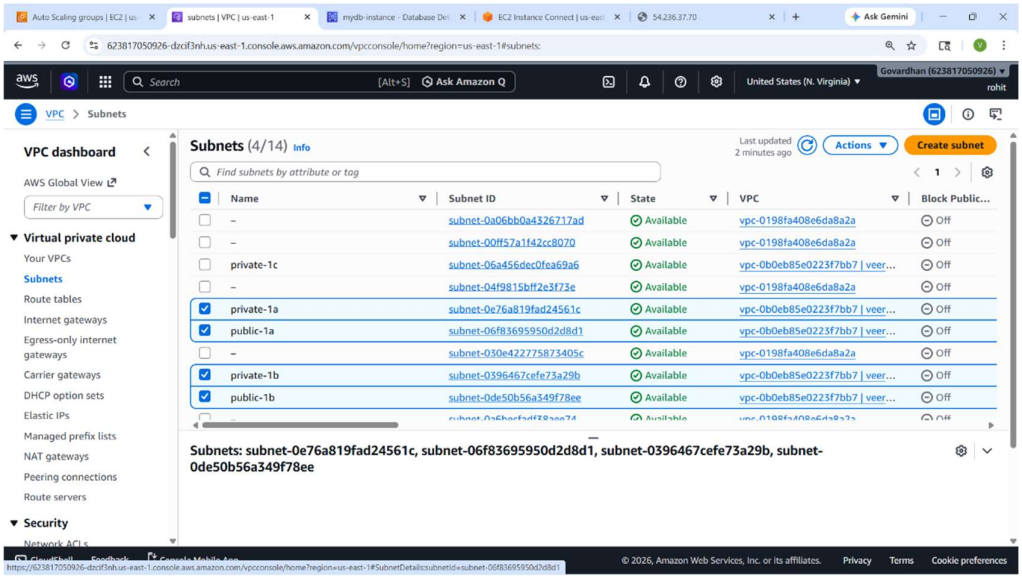
Step 4: Internet Gateway

An Internet Gateway is attached to the VPC for external connectivity.
It allows instances in public subnets to communicate over the internet.
This is essential for services like public-facing EC2 instances.
The gateway is tagged for better visibility in the AWS console.



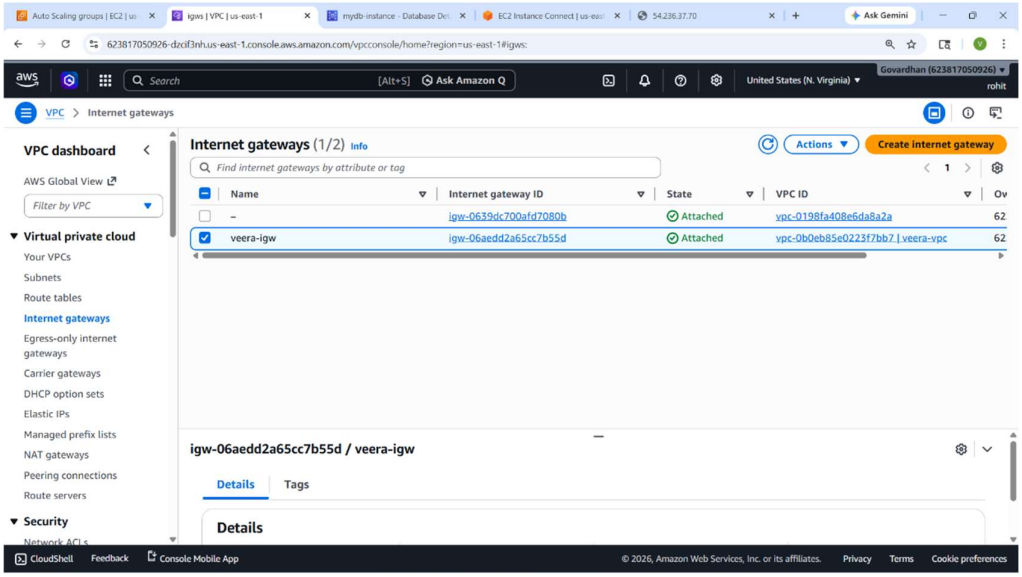
Step 5: NAT Gateway and EIP

An Elastic IP is created and used to launch a NAT Gateway.
The NAT Gateway is placed in a public subnet to route internet traffic.
It allows private subnet instances to access the internet securely. This setup protects private resources from direct exposure.



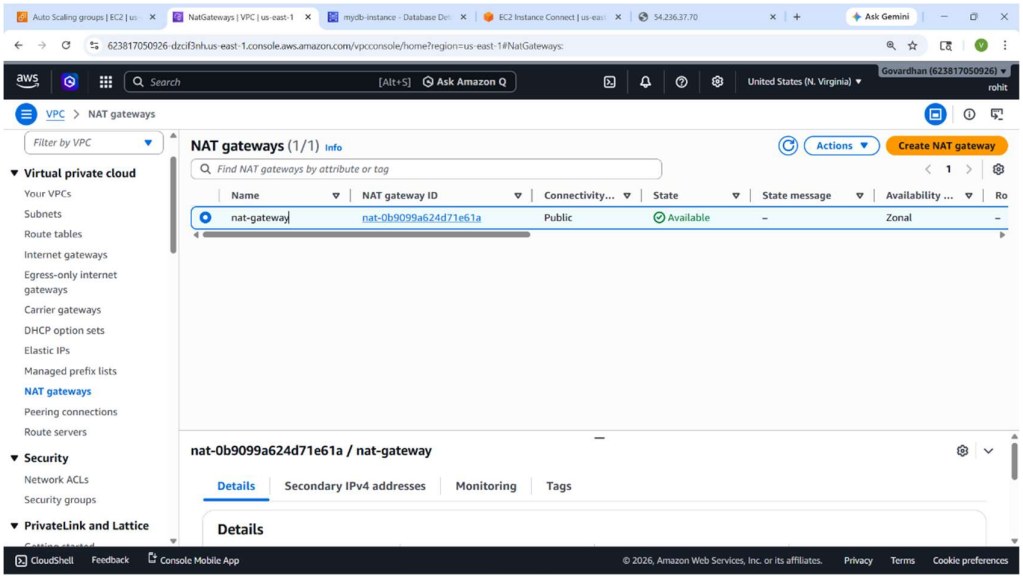
Step 6: Route Tables and Associations

Two route tables are configured: one for public and one for private subnets.
The public route sends internet traffic through the Internet Gateway.
The private route uses the NAT Gateway for outbound internet access. Each subnet is associated with its appropriate route table.



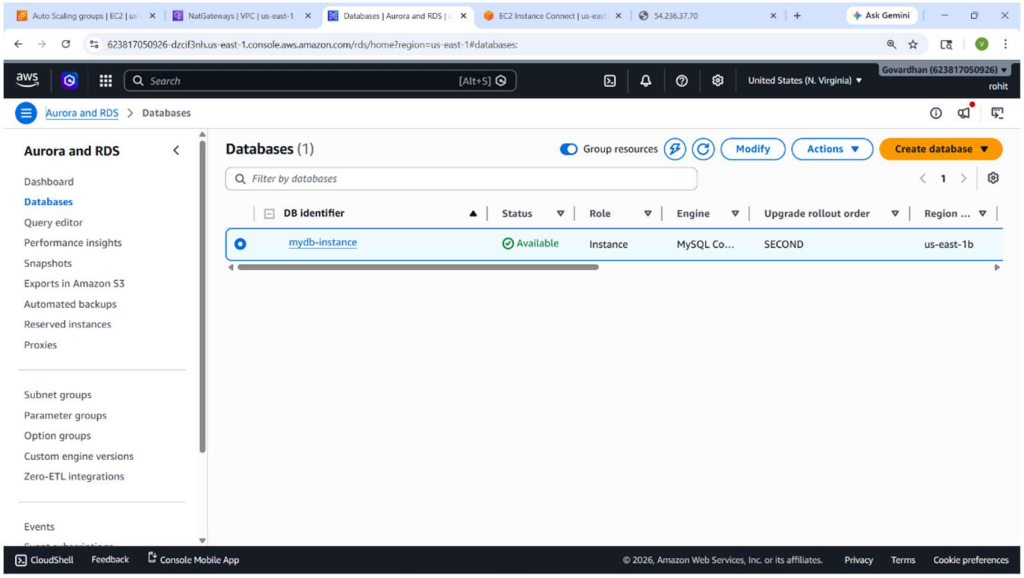
Step 7: Security Groups

Three security groups are defined to control traffic.
Public SG allows SSH (22) and HTTP (80) from anywhere.
Private SG allows unrestricted internal traffic within the VPC.
RDS SG allows MySQL access (3306) only from the private EC2 range.



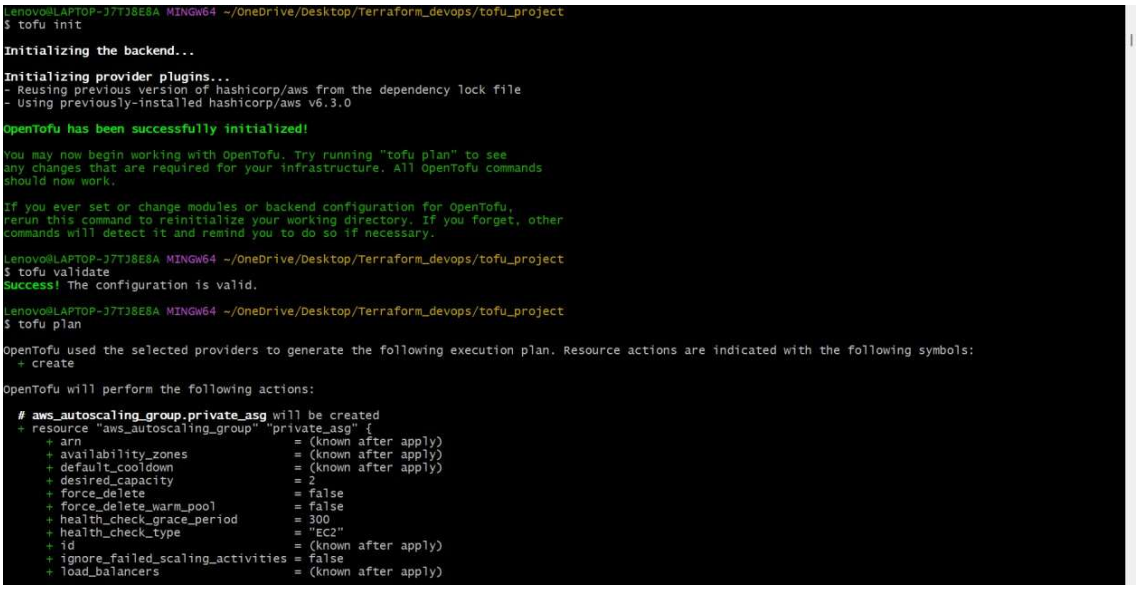
Step 8: Key Pair

A key pair named terraform-key is created for SSH access.
This key is linked to EC2 instances to allow secure login.
Only users with the private key can access the instances. This enhances security for remote server management.



Step 9: Launch Templates

Two launch templates define instance configurations.
Public launch template uses the public security group.
Private launch template is assigned to internal workloads. Both templates include AMI, instance type, and key pair.



Step 10: Public Load Balancer

An Application Load Balancer is deployed in the public subnets.
It distributes HTTP traffic across EC2 instances.
This helps achieve better performance and fault tolerance.
It is internet-facing and secured by the public security group.

Step 12: Auto Scaling Groups

Two Auto Scaling Groups (ASGs) are created for public and private tiers.
Each group automatically maintains desired EC2 capacity.
The ASGs use the launch templates and are tied to subnets and target groups.
Scaling ensures high availability and resource optimization.

Step 11: Private Load Balancer

A second load balancer is created, but internal only.
It routes traffic to private EC2 instances in private subnets.
Useful for microservices or internal APIs without public exposure.
This setup ensures controlled internal communication.

Step 13: Auto Scaling Policies

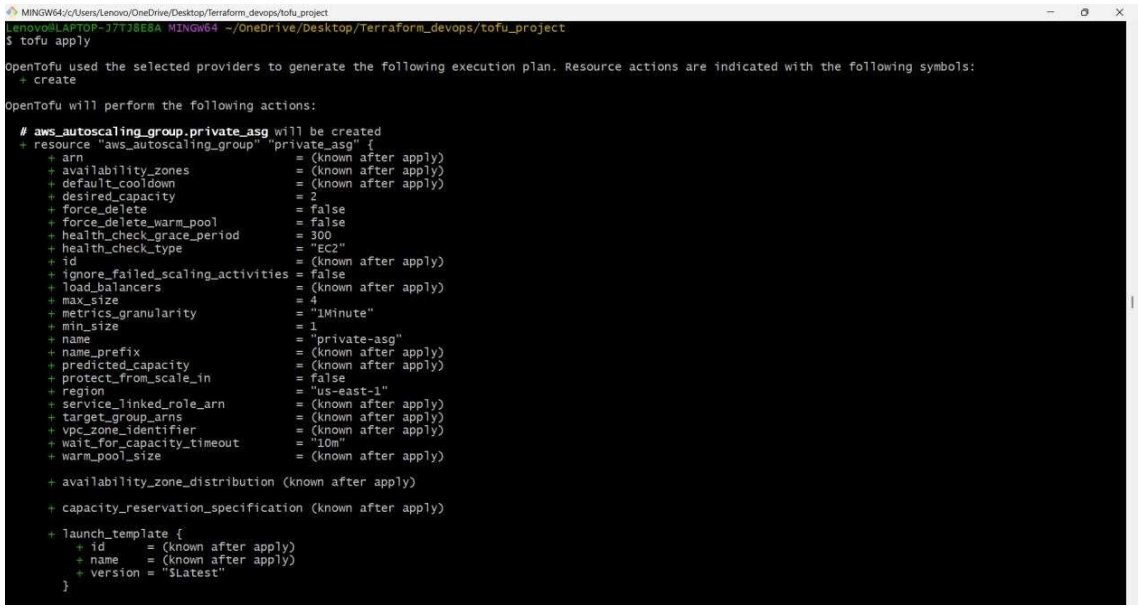
Scaling policies are added to increase instance count when needed.
The adjustment type is based on capacity changes.
This allows the system to handle increased load dynamically. Cooldown
settings help prevent rapid scaling actions.

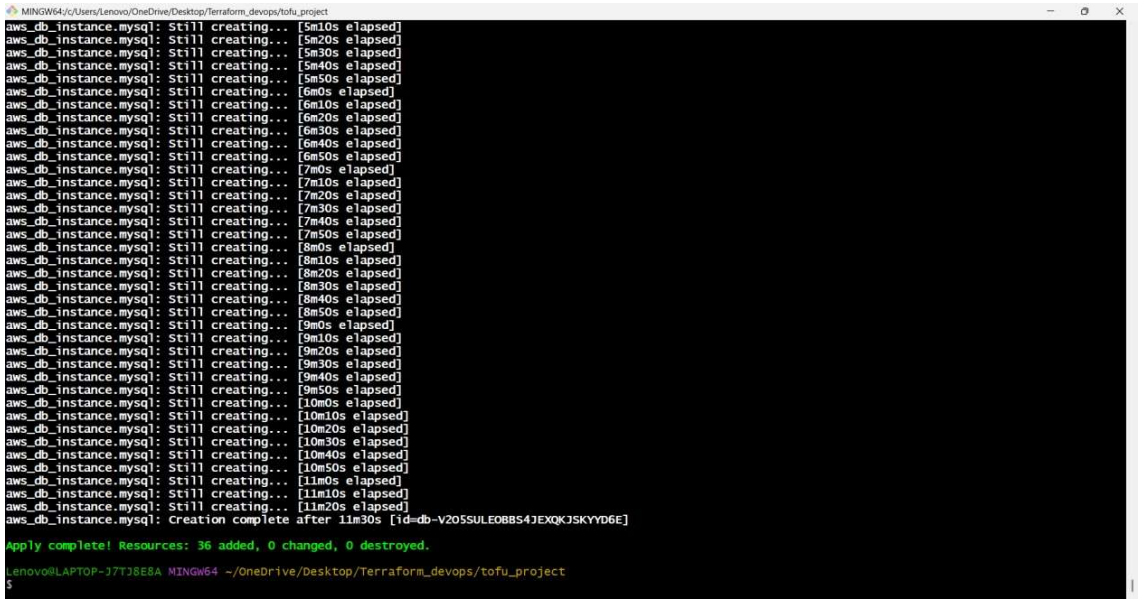
Step 14: RDS Subnet Group

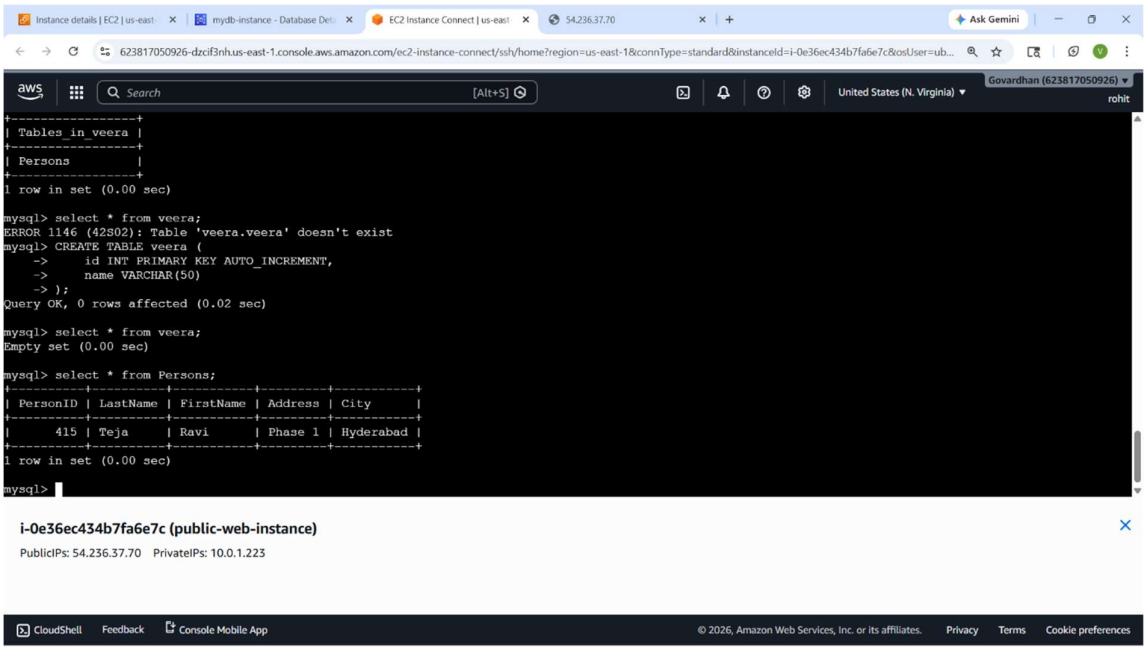
A subnet group is created for the RDS instance deployment.
It includes the two private RDS subnets for high availability.
This group ensures the database is launched in a secure zone.
Subnet groups are mandatory for RDS in VPC environments.

Step 15: RDS MySQL Instance

A managed MySQL database is provisioned using RDS.
It is deployed in private subnets and not publicly accessible.
Multi-AZ is enabled for automatic failover and high availability. The database is secured using the RDS-specific security group.







Terraform code :- terraform

{ required_providers { aws

= { source =

"hashicorp/aws" version =

"6.3.0"

}

}

}

provider "aws" {

region = "us-east-1"

}

VPC resource "aws_vpc"

"main" { cidr_block =

"10.0.0.0/16"

enable_dns_hostnames = true

```
enable_dns_support   = true   tags
= { Name = "main-vpc" }
}

# Subnets - 6 total resource
"aws_subnet" "public_a" {   vpc_id
= aws_vpc.main.id   cidr_block
= "10.0.1.0/24"   availability_zone
= "us-east-1a"
map_public_ip_on_launch = true
tags = { Name = "public-a" }

}

resource "aws_subnet" "public_b" {
vpc_id            = aws_vpc.main.id
cidr_block        = "10.0.2.0/24"
availability_zone = "us-east-1b"
map_public_ip_on_launch = true   tags
= { Name = "public-b" }
}

resource "aws_subnet" "private_ec2_a" {
vpc_id            = aws_vpc.main.id
cidr_block        = "10.0.3.0/24"
availability_zone = "us-east-1a"   tags =
{ Name = "private-ec2-a" }
}
```

```
resource "aws_subnet" "private_ec2_b" {
vpc_id            = aws_vpc.main.id
cidr_block        = "10.0.4.0/24"
availability_zone = "us-east-1b"   tags =
{ Name = "private-ec2-b" }
}

resource "aws_subnet" "private_rds_a" {
vpc_id            = aws_vpc.main.id
cidr_block        = "10.0.5.0/24"
availability_zone = "us-east-1a"   tags =
{ Name = "private-rds-a" }

}

resource "aws_subnet" "private_rds_b" {
vpc_id            = aws_vpc.main.id
cidr_block        = "10.0.6.0/24"
availability_zone = "us-east-1b"   tags =
{ Name = "private-rds-b" }
}

# Internet Gateway resource
"aws_internet_gateway" "igw" {   vpc_id
= aws_vpc.main.id   tags = { Name =
"main-igw" }
}

# Elastic IP for NAT resource
"aws_eip" "nat" {
```

```
}

# NAT Gateway (in public subnet A)
resource "aws_nat_gateway" "nat" {
allocation_id = aws_eip.nat.id   subnet_id
= aws_subnet.public_a.id   tags = { Name =
"main-nat" }   depends_on =
[aws_internet_gateway.igw]
}

# Route Tables resource
"aws_route_table" "public" {   vpc_id
= aws_vpc.main.id   route {
cidr_block = "0.0.0.0/0"
gateway_id =
aws_internet_gateway.igw.id
  }
  tags = { Name = "public-rt" }
}

resource "aws_route_table" "private" {
vpc_id = aws_vpc.main.id   route {
cidr_block     = "0.0.0.0/0"
nat_gateway_id = aws_nat_gateway.nat.id
  }
  tags = { Name = "private-rt" }
}
```

```
# Route Table Associations resource
"aws_route_table_association" "public_a" {
subnet_id      = aws_subnet.public_a.id
route_table_id = aws_route_table.public.id
} resource "aws_route_table_association" "public_b"
{   subnet_id      = aws_subnet.public_b.id
route_table_id = aws_route_table.public.id
} resource "aws_route_table_association" "private_ec2_a"
{   subnet_id      = aws_subnet.private_ec2_a.id
route_table_id = aws_route_table.private.id

} resource "aws_route_table_association" "private_ec2_b"
{   subnet_id      = aws_subnet.private_ec2_b.id
route_table_id = aws_route_table.private.id
} resource "aws_route_table_association" "private_rds_a"
{   subnet_id      = aws_subnet.private_rds_a.id
route_table_id = aws_route_table.private.id
} resource "aws_route_table_association" "private_rds_b"
{   subnet_id      = aws_subnet.private_rds_b.id
route_table_id = aws_route_table.private.id
}

# Security Groups resource
"aws_security_group" "public_sg" {    name
=  "public-sg"        description  =  "Allow
HTTP/SSH"   vpc_id      = aws_vpc.main.id
ingress {    from_port  = 22    to_port    = 22
protocol       =  "tcp"          cidr_blocks  =
["0.0.0.0/0"]
```

```
  }   ingress {
from_port   = 80
to_port     = 80
protocol    = "tcp"
cidr_blocks =
["0.0.0.0/0"]
  }   egress {     from_port   =
0     to_port     = 0
protocol    = "-1"
cidr_blocks = ["0.0.0.0/0"]
  }
  tags = { Name = "public-sg" }
}

resource "aws_security_group" "private_sg" {
name       = "private-sg"   description = "Allow
internal traffic"   vpc_id      = aws_vpc.main.id
ingress {      from_port   = 0      to_port     = 0
protocol          =   "-1"             cidr_blocks   =
["10.0.0.0/16"]
  }   egress {     from_port   =
0     to_port     = 0
protocol    = "-1"
cidr_blocks = ["0.0.0.0/0"]
  }
  tags = { Name = "private-sg" }
}
```

```
resource "aws_security_group" "rds_sg" {  name
= "rds-sg"  description = "Allow MySQL from
private EC2"  vpc_id  = aws_vpc.main.id
ingress {  from_port = 3306  to_port  = 3306
protocol  = "tcp"  cidr_blocks = ["10.0.0.0/16"]
 }  egress {  from_port  =
0  to_port  = 0
protocol  = "-1"
cidr_blocks = ["0.0.0.0/0"]
 }
 tags = { Name = "rds-sg" }
}

# Key Pair resource "aws_key_pair"
"deployer" {  key_name  =
"terraform-key"
 public_key = "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQD3F6tyPEFEzV0LX3X8BsXdMsQz1x2cEi
kKDEY0aIj41qgxMCP/iteneqXSIFZBp5vizPvaoIR3Um9xK7PGoW8giupGn+EPuxIA4cD
M4vzOqOkiMPhz5XK0whEjkVzTo4+S0puvDZuwIsdiW9mxhJc7tgBNL0cYlWSYVkz4G/f
slNfRPW5mYAM49f4fhtxPb5ok4Q2Lg9dPKVHO/Bgeu5woMc7RY0p1ej6D4CKFE6lymS
DJpW0YHX/wqE9+cfEauh7xZcG0q9t2ta6F6fmX0agvpFyZo8aFbXeUBr7osSCJNgvavWb
M/06niWrOvYX2xwWdhXmXSrbX8ZbabVohBK41"
}

# Launch Templates resource "aws_launch_template"
"public_lt" {  name_prefix  = "public-lt-"  image_id  = "ami-
020cba7c55df1f615"  instance_type = "t3.micro"  key_name
= aws_key_pair.deployer.key_name  vpc_security_group_ids =
[aws_security_group.public_sg.id]  tag_specifications  {
```

```
resource_type = "instance"       tags = { Name = "public-asg-
instance" }
  }
}

resource "aws_launch_template" "private_lt" {  name_prefix  =
"private-lt-"      image_id            =  "ami-020cba7c55df1f615"
instance_type   =   "t3.micro"          key_name                  =
aws_key_pair.deployer.key_name    vpc_security_group_ids =
[aws_security_group.private_sg.id]         tag_specifications   {
resource_type = "instance"      tags = { Name = "private-asg-
instance" }
  }
}

# Public Load Balancer resource
"aws_lb" "public_alb" { name
= "public-alb" internal          =
false

  load_balancer_type = "application" security_groups    =
[aws_security_group.public_sg.id] subnets           =
[aws_subnet.public_a.id, aws_subnet.public_b.id]
enable_deletion_protection = false   tags = { Name = "public-alb" }
}

resource "aws_lb_target_group" "public_tg" {
name     = "public-tg"   port     = 80   protocol
= "HTTP"   vpc_id   = aws_vpc.main.id
}
```

```
resource "aws_lb_listener" "public_listener" {
load_balancer_arn = aws_lb.public_alb.arn
  port              = 80   protocol          = "HTTP"
default_action {     type              = "forward"
target_group_arn = aws_lb_target_group.public_tg.arn
  }
}

# Private Load Balancer resource
"aws_lb" "private_alb" { name
= "private-alb" internal           =
true load_balancer_type =
"application" security_groups    =
[aws_security_group.private_sg.i
d] subnets            =
[aws_subnet.private_ec2_a.id,
aws_subnet.private_ec2_b.id]
enable_deletion_protection =
false tags = { Name = "private-
alb" }
}

resource "aws_lb_target_group" "private_tg" {
name     = "private-tg"   port     = 80   protocol
= "HTTP"   vpc_id   = aws_vpc.main.id
}
```

```
resource "aws_lb_listener" "private_listener" {
load_balancer_arn = aws_lb.private_alb.arn
  port              = 80   protocol          = "HTTP"
default_action {     type             = "forward"
target_group_arn = aws_lb_target_group.private_tg.arn
  }
}

# Auto Scaling Groups resource
"aws_autoscaling_group" "public_asg" {
  name                      = "public-asg"
  max_size                  = 4 min_size
  = 1 desired_capacity              = 2
  vpc_zone_identifier                 =
  [aws_subnet.public_a.id,
  aws_subnet.public_b.id]
  health_check_type          = "EC2"
  health_check_grace_period = 300
  target_group_arns                   =
  [aws_lb_target_group.public_tg.a
  rn]

  launch_template {     id      =
aws_launch_template.public_lt.id
version = "$Latest"
  }
```

```
  tag {     key                 = "Name"
value               = "public-asg-instance"
propagate_at_launch = true
  }

  lifecycle {
create_before_destroy = true
  }
}

resource "aws_autoscaling_group" "private_asg" {
  name                      = "private-asg"    max_size                  = 4    min_size                  = 1
desired_capacity          = 2 vpc_zone_identifier       = [aws_subnet.private_ec2_a.id,
aws_subnet.private_ec2_b.id] health_check_type         = "EC2"
  health_check_grace_period = 300 target_group_arns         =
  [aws_lb_target_group.private_tg.arn]

  launch_template {     id      =
aws_launch_template.private_lt.id
version = "$Latest"
  }

  tag {     key                 = "Name"
value               = "private-asg-instance"
propagate_at_launch = true
  }

  lifecycle {
create_before_destroy = true
```

```
  }
}

# Auto Scaling Policies resource
"aws_autoscaling_policy" "public_scale_up" {
  name                   = "public-scale-up"
  autoscaling_group_name = aws_autoscaling_group.public_asg.name
scaling_adjustment     = 1    adjustment_type        =
"ChangeInCapacity"    cooldown               = 300
}

resource "aws_autoscaling_policy" "private_scale_up" {
  name                   = "private-scale-up"
  autoscaling_group_name = aws_autoscaling_group.private_asg.name
scaling_adjustment     = 1 adjustment_type        = "ChangeInCapacity"
cooldown               = 300
}

# RDS Subnet Group resource "aws_db_subnet_group" "rds" {   name
= "main-rds-subnet-group"    subnet_ids = [aws_subnet.private_rds_a.id,
aws_subnet.private_rds_b.id]    tags = { Name = "main-rds-subnet-group"
}
}

# RDS Instance resource "aws_db_instance" "mysql" {
identifier             = "main-mysql-db"    engine                 =
"mysql"   engine_version         = "8.0"   instance_class         =
"db.t3.micro"   allocated_storage      = 20   username             =
"admin"       password                              =  "9347980893"
```

```
db_subnet_group_name      = aws_db_subnet_group.rds.name
vpc_security_group_ids    =  [aws_security_group.rds_sg.id]
skip_final_snapshot     = true   publicly_accessible     = false
multi_az                = true
  tags = { Name = "main-mysql-db" }
}
```